

SPEC — Banco POS: Receiving Flow (v1, build-ready)

Back-office receiving for a single head shop. **No counts, no MRP — dates and velocity only.** Banco is the product source of truth; the vending system (Harvin) is left alone in v1. This supersedes the earlier receiving stub.

Goal

A back-room **Receiving mode**: a delivery box → every product entered fast and clean, in calm time, no customer waiting. Each product is **born once** here as the clean master record that then serves

the counter, prints its own label, and (later) feeds velocity reporting. Handles three situations: known restock, first-time barcoded item, uncoded item needing an internal label. Reuses the existing scanner, add-product form, and WeasyPrint.

Philosophy, so the agent does not "helpfully" add inventory: dates and velocity, never counts. No quantity-on-hand, no MRP, anywhere.

Context for the agent

- Stack: FastAPI + Jinja2, Caddy, Postgres. Already working: camera scanner (html5-qrcode), tier-3 add-product form, WeasyPrint PDF/print, a sales/ring-up log.

- Receiving mode = the existing add-product form, driven in a loop, plus: a supplier session default, save-and-next, an optional AI photo-draft step, and a batched label queue.
 - **Do not** build P2P, supplier master, goods-receipt reconciliation, stock counts, vending slots, or MRP. See OUT OF SCOPE.
-

Product master (v1 fields)

Static only — true-once, never drift:

- `barcode` — manufacturer EAN/UPC, or generated internal Code128/QR
- `name`
- `price` — sell price
- `cost` — what he paid (enables margin + margin-weighted velocity later;

static,

updated only when it changes)

- `category` – default: last used this session
- `unit_of_measure` – default "each"
- `photo` – optional; show a "no image available" placeholder if none
- `supplier` – session default; free-text + autocomplete; **not** an entity
- `location` – optional free text (e.g. "front display", "top shelf"); static, for restock guidance only
- `vending_eligible` – boolean flag, **informational only**; does NOT integrate
Harvin in v1
- `last_received` – date, set on save
- `notes` – freeform escape hatch

Not stored: quantity on hand,
min/max/reorder thresholds, PO, supplier
order

history, stock movements, vending slot data.

Receiving session

1. Start Receiving → prompt **once** for supplier (optional) → pre-fills the supplier field for every item this session.
 2. Item loop, **save-and-next**: after each save the form clears and reloads for the next item. Never returns to the till, never makes the operator hunt for an "add another" button.
-

Item loop — three branches (scan each item)

A. Known item (restock — the common case after the first order)

Scanner matches an existing product →
show name/price/photo → operator
confirms →
save-and-next. Updates `last_received`.
No quantity.

B. Barcoded, not yet in the system (first time stocked)

"Not on file" → add form opens, barcode
pre-filled → operator optionally snaps a
photo → **AI drafts**
name/description/category from the
photo → operator
reviews/edits → confirm → save-and-
next. No label (it has a manufacturer
barcode).

C. Uncoded item (no manufacturer barcode)

Tap "**generate internal barcode**" →
Code128 (or QR — see symbology)
created and
filled → photo optional → AI draft

optional → set **label count** (default 1) → label(s) added to the print **queue** → save-and-next.

AI note: AI works from the **photo**, never the barcode number (a barcode is just digits). Flow is scan → photo → AI drafts → human confirms. **Never advertise**

"scan auto-fills." If there's no photo or AI is unavailable, the operator types manually — AI is an accelerator, never a dependency.

Labels — batched queue

- Labels only for branch C (uncoded items).
- During a session, each label + its count (20 trays → 20) goes into a **label queue** — not printed one at a time.

- "Print labels" renders the whole queue packed **N-up on a standard sheet** (e.g. up to 64 per A4) → one page, cut once. Never one-label-per-page.
- Label content: barcode/QR + shop name ("Artemis") + product name. **No price** (price changes must not force re-labeling).
- The app only ever generates a label **PDF** — it does not care what it prints on. So:
 - **Demo / zero-spend:** print on plain paper, cut out, removable transparent tape over the code. (Works — proven.)
 - **Adopted:** ~€100 Brother/Dymo thermal, die-cut roll labels — the **same**

queue streams to the roll, no redesign.

- **Symbology – decide from the dry-run:** default Code128 (works with the existing phone scanner and any future USB gun). If small labels scan unreliably on the phone (AC5), switch internal labels to QR – phones decode QR more forgivingly at small sizes.
-

Preconditions

- HTTPS – satisfied.
 - A printer reachable from the receiving device (browser print is fine; no integration). **No hardware purchase needed to build or dry-run.**
-

OUT OF SCOPE – DO NOT BUILD

-  Quantity on hand / counts / decrement / sentinel values.
-  MRP, min/max, reorder thresholds.
-  Purchase orders / procure-to-pay / three-way match / delivery-vs-PO reconciliation.
-  Supplier as an entity (flat field + autocomplete only).
-  Vending slot management / Harvin integration / any multi-channel sales-org machinery.
-  Stock-movement ledger.
-  Mass/bulk edit of existing products.

The flow does ONE thing: get each delivered product to **exist** – named, priced,

cost-known, categorised, photographed-or-placeholdered, labeled if uncoded, scannable — fast, in calm time.

Acceptance criteria (manual, real device + printer)

- **AC1:** supplier set once → pre-fills every item that session.
- **AC2 (A):** known scan → shows → confirm → next; `last_received` updated; no quantity stored.
- **AC3 (B):** unknown manufacturer barcode → add form, barcode pre-filled → photo → AI draft appears → edit → save → next; no label.
- **AC4 (C):** uncoded → generate internal code → details → label count 20 → 20

labels enter the queue → save → next;
no count stored.

- **AC5:** print the queue → labels render N-up on one sheet → a printed label scans on the phone and rings up at the till.
 - **AC6:** defaults carry (category = last used; UoM = "each").
 - **AC7:** no stock quantity reported anywhere (nothing leaked in).
 - **AC8:** AI unavailable → operator still completes entry manually, no blocking.
-

Dry-run walkthroughs

1. **Restock papers (known EAN):**
receiving → supplier "Acme" → scan one →
known → confirm → ~5s. Does **not** scan all 50.

2. **First-time barcoded grinder:** scan → not on file → photo → AI drafts "BrandX Grinder / Accessories" → fix price 24.90 → save → ~30s, once ever.
 3. **Uncoded trays ×20:** generate code → photo/AI → name/price → label count 20 → queue → at session end print one A4 sheet of labels, cut, tape → each tray scans at the till. The "20" is gone after printing.
-

Implementation order

1. Receiving entry + supplier session prompt (free-text + autocomplete from prior suppliers).
2. Save-and-next loop around the existing add-product form; add `cost`,

`location`, `vending_eligible`,
`last_received` to the product
master.

3. Branch logic: known → confirm;
unknown barcode → add; "generate
internal
barcode" → Code128/QR + add.
 4. AI photo-draft step (photo → vision
draft → editable fields → confirm);
graceful skip if unavailable.
 5. Label queue + N-up PDF via
WeasyPrint (`python-barcode`
Code128 / `qrcode` for
QR); **no price on label**.
 6. Dry-run AC1–AC8; decide Code128 vs
QR from AC5.
-

**Appendix — what this
enables next (SEPARATE
spec — DO NOT build here)**

Receiving captures `last_received` and `cost` ; the sales/ring-up log already exists. Those feed a later **velocity reporting** build: top sellers, dead stock (no sale in N days), gap-based reorder hints, velocity change vs 4-week average, time-of-day / day-of-week curves, margin-weighted ranking, and a draft per-supplier shopping list. Captured here only so the data model is right now — **the reporting is its own spec, not part of receiving.**